

Docker + FastAPI 零基础学习指南

面向 Multi-Agent 项目的基础技术栈学习

Docker + FastAPI 零基础学习指南

面向 Multi-Agent 项目的基础技术栈学习

前置条件：Docker 和 FastAPI 已安装完毕

目录

第一部分：Docker 基础

- [Docker 核心概念：镜像、容器、Dockerfile](#1-docker-核心概念)
- [Docker 基本命令](#2-docker-基本命令)
- [编写你的第一个 Dockerfile](#3-编写你的第一个-dockerfile)
- [实战：把 Python 应用打包成 Docker 镜像](#4-实战打包-python-应用)

第二部分：Docker Compose（重点）

- [什么是 Docker Compose](#5-什么是-docker-compose)
- [docker-compose.yml 逐行解析](#6-docker-compose.yml-逐行解析)
- [实战：用 docker-compose 启动 PostgreSQL + Redis](#7-实战-postgresql--redis)

第三部分：FastAPI 基础

- [FastAPI 是什么](#8-fastapi-是什么)
- [第一个 FastAPI 应用](#9-第一个-fastapi-应用)
- [路径参数和查询参数](#10-路径参数和查询参数)
- [请求体与 Pydantic 模型](#11-请求体与-pydantic-模型)
- [异步编程 async/await](#12-异步编程-asyncawait)
- [依赖注入 Depends](#13-依赖注入-depends)
- [自动文档 Swagger UI](#14-自动文档-swagger-ui)

第四部分：综合实战

- [实战项目：完整的 CRUD API（FastAPI + PostgreSQL + Docker Compose）](#15-综合实战完整项目)

第一部分：Docker 基础

1. Docker 核心概念

1.1 类比理解

想象你要开一家餐厅：

镜像 (Image)	菜谱	一个只读模板，包含了运行应用需要的一切：代码、运行环境、依赖库
容器 (Container)	按菜谱做出来的一道菜	镜像的运行实例。一个镜像可以运行出多个容器
Dockerfile	菜谱的制作步骤	一个文本文件，一步步描述如何构建镜像
仓库 (Registry)	菜谱商店	存放镜像的地方，最常用的是 Docker Hub (hub.docker.com)
Volume（数据卷）	保鲜柜	持久化存储，容器删除后数据还在
Network（网络）	厨房内部通道	让容器之间能互相通信

1.2 关键理解

镜像 = 一个打包好的"快照"，包含应用 + 所有依赖

容器 = 镜像运行起来后的"活"的进程

同一个镜像可以启动多个容器，就像同一道菜谱可以做多份菜。

2. Docker 基本命令

2.1 拉取镜像

```
[ bash ]
```

```
# 从 Docker Hub 下载一个镜像
docker pull nginx:latest
```

```
# 指定版本拉取
docker pull nginx:1.25
docker pull postgres:17
docker pull redis:7.4-alpine
```

解释：

- - **nginx** 是镜像名称
- - **:latest** 是标签（tag），表示最新版本，可以省略
- - **:1.25** 表示指定版本
- - **alpine** 表示基于 Alpine Linux，体积更小

2.2 运行容器

```
[ bash ]
```

```
# 最简单的运行方式
```

```
docker run nginx
```

```
# 后台运行 (推荐)
```

```
docker run -d nginx
```

```
# 端口映射: 把容器的 80 端口映射到本机的 8080 端口
```

```
docker run -d -p 8080:80 nginx
```

```
# 给容器起个名字
```

```
docker run -d -p 8080:80 --name my-nginx nginx
```

```
# 设置环境变量
```

```
docker run -d -e POSTGRES_PASSWORD=mysecret --name my-postgres postgres:17
```

解释 **-p 8080:80**:

- - 格式是 **主机端口:容器端口**
- - 左边 **8080** 是你本机访问的端口
- - 右边 **80** 是容器内部服务监听的端口
- - 意思: 访问 **http://localhost:8080** 就等于访问容器内的 **http://localhost:80**

2.3 查看容器

```
[ bash ]
```

```
# 查看正在运行的容器
```

```
docker ps
```

```
# 查看所有容器 (包括已停止的)
```

```
docker ps -a
```

```
# 查看容器日志 (非常重要! 调试时必用)
```

```
docker logs my-nginx
```

```
# 实时跟踪日志
```

```
docker logs -f my-nginx
```

2.4 停止和删除容器

```
[ bash ]
```

```
# 停止容器
```

```
docker stop my-nginx
```

```
# 启动已停止的容器
```

```
docker start my-nginx
```

```
# 删除容器 (必须先停止)
```

```
docker rm my-nginx
```

```
# 停止并删除一步到位
```

```
docker rm -f my-nginx
```

2.5 进入容器内部

```
[ bash ]
```

```
# 进入容器的 shell (像在容器里开了一个终端)
```

```
docker exec -it my-nginx /bin/bash
```

```
# 有些容器没有 bash, 用 sh
docker exec -it my-nginx /bin/sh

# 退出容器
exit
```

2.6 查看镜像

```
[ bash ]

# 列出本地所有镜像
docker images

# 删除镜像
docker rmi nginx

# 删除所有未使用的镜像
docker image prune -a
```

3. 编写你的第一个 Dockerfile

3.1 Dockerfile 是什么

Dockerfile 是一个没有扩展名的文本文件，内容是一步步的指令，告诉 Docker 如何构建镜像。

3.2 一个最简单的例子

```
[ dockerfile ]

# 第 1 行: 基于哪个镜像来构建 (必须的第一行)
FROM python:3.12-slim

# 第 2 行: 设置工作目录 (容器内的目录)
WORKDIR /app

# 第 3 行: 复制文件到容器中
COPY requirements.txt .

# 第 4 行: 在容器中运行命令
RUN pip install --no-cache-dir -r requirements.txt

# 第 5 行: 把应用代码复制进来
COPY . .

# 第 6 行: 声明容器要监听的端口 (只是声明, 不做实际映射)
EXPOSE 8000

# 第 7 行: 容器启动时运行的命令
CMD ["python", "main.py"]
```

3.3 每行指令详解

FROM	指定基础镜像	"基于 Ubuntu 系统来搭建"
WORKDIR	设置工作目录	先 cd 到某个目录
COPY	从本机复制文件到容器	把文件拷贝进去

RUN	构建时运行的命令	安装依赖、创建目录等
EXPOSE	声明端口	"这个服务用 8000 端口"
CMD	容器启动时运行的命令	入口程序
ENV	设置环境变量	设置 PATH、DATABASE_URL 等

3.4 构建镜像

[bash]

```
# 在 Dockerfile 所在目录运行
docker build -t my-app:1.0 .
```

```
# 解释:
# -t my-app:1.0 给镜像打标签, 名字是 my-app, 版本是 1.0
# . 表示当前目录是构建上下文 (Docker 会把这个目录的所有文件发给 Docker 引擎)
```

3.5 运行构建好的镜像

[bash]

```
docker run -d -p 8000:8000 --name my-running-app my-app:1.0
```

4. 实战：打包 Python 应用

4.1 准备一个最简单的 Python Web 应用

创建以下文件：

hello_app/main.py

[python]

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(b"<h1>Hello from Docker!</h1>")

if __name__ == "__main__":
    server = HTTPServer(("0.0.0.0", 8000), Handler)
    print("Server running on port 8000")
    server.serve_forever()
```

hello_app/Dockerfile

[dockerfile]

```
FROM python:3.12-slim
WORKDIR /app
COPY main.py .
EXPOSE 8000
CMD ["python", "main.py"]
```

4.2 构建和运行

[bash]

```
# 进入 hello_app 目录
cd hello_app

# 构建镜像
docker build -t hello-app:1.0 .

# 运行容器
docker run -d -p 8000:8000 --name hello hello-app:1.0

# 验证: 浏览器打开 http://localhost:8000
# 或者用 curl
curl http://localhost:8000

# 查看日志
docker logs hello

# 停止并清理
docker stop hello
docker rm hello
```

到这里你就掌握了 Docker 的基础。但实际项目中我们很少一个个 `docker run`，而是用 `docker-compose` 来管理多个服务。接下来是最重要的部分。

第二部分：Docker Compose（重点）

5. 什么是 Docker Compose

5.1 为什么需要它

你的 Multi-Agent 项目需要同时运行这些服务：

```
FastAPI 应用 —→ PostgreSQL 数据库
                —→ Redis 缓存
                —→ Milvus 向量数据库
                —→ Ollama AI 服务
```

如果不用 `docker-compose`，你需要手动运行 5 条 `docker run` 命令，还要手动配置网络让它们互相连通。

`docker-compose` 的作用：用一个 YAML 文件定义所有服务，然后一条命令全部启动。

```
[ bash ]
```

```
docker-compose up -d # 启动所有服务（后台运行）
docker-compose down  # 停止并删除所有服务
```

5.2 docker-compose 文件位置

```
你的项目/
├── docker-compose.yml ← 这个文件
└── ...
```

6. docker-compose.yml 逐行解析

6.1 最简单的例子：只运行一个服务

[yaml]

```
# docker-compose.yml
version: "3.8"      # 版本声明（可以省略，较新版本不强制要求）

services:           # 所有服务定义在这里
  web:              # 服务名称（自定义）
    image: nginx:latest # 使用什么镜像
    ports:
      - "8080:80"     # 端口映射：主机8080 → 容器80
```

运行：

[bash]

```
docker-compose up -d
# 这会启动：拉取 nginx 镜像 → 创建容器 → 启动服务
```

6.2 多服务例子：FastAPI + PostgreSQL

[yaml]

```
services:
  # 服务 1: FastAPI 应用
  fastapi-app:
    image: my-fastapi-app:latest
    ports:
      - "8000:8000"      # 主机 8000 映射到容器 8000
    environment:         # 环境变量
      - DATABASE_URL=postgresql://user:password@db:5432/mydb
      - REDIS_URL=redis://redis:6379/0
    depends_on:          # 依赖关系：等 db 启动后再启动 fastapi-app
      - db
      - redis

  # 服务 2: PostgreSQL 数据库
  db:
    image: postgres:17
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=mydb
    volumes:
      - postgres_data:/var/lib/postgresql/data # 数据持久化

  # 服务 3: Redis 缓存
  redis:
    image: redis:7.4-alpine
    ports:
      - "6379:6379"

# 声明要使用的数据卷
volumes:
  postgres_data: # Docker 会自动创建这个数据卷
```

6.3 关键字段详解

services:

- - 这是 docker-compose 的核心，每个子键是一个服务

- - 服务名称（如 **fastapi-app**、**db**、**redis**）是容器间互相访问的主机名
- - 比如 FastAPI 访问数据库时，连接字符串写 **postgresql://user:password@db:5432/mydb**，这里的 **@db** 就是服务名

image vs build:

[yaml]

```
# 使用已有镜像
image: postgres:17

# 从本地 Dockerfile 构建
build: .

# 从指定目录的 Dockerfile 构建
build: ./fastapi-app
```

ports:

[yaml]

```
ports:
  - "主机端口:容器端口"
  - "8000:8000" # 访问 localhost:8000 → 容器内 8000 端口
  - "5432:5432" # 访问 localhost:5432 → 容器内 PostgreSQL
```

environment:

[yaml]

```
# 方式一：列表格式
environment:
  - KEY=value
  - DATABASE_URL=postgresql://user:password@db:5432/mydb

# 方式二：字典格式（推荐，更清晰）
environment:
  DATABASE_URL: postgresql://user:password@db:5432/mydb
  REDIS_URL: redis://redis:6379/0
```

depends_on:

[yaml]

```
depends_on:
  - db
  - redis
```

- - 控制启动顺序：先启动 **db** 和 **redis**，再启动当前服务
- - 注意：只保证启动顺序，不保证数据库"就绪"（数据库可能正在初始化）

volumes:

[yaml]

```
# 命名卷（推荐用于数据库持久化）
volumes:
  - postgres_data:/var/lib/postgresql/data

# 绑定挂载（开发时方便改代码，改完容器立即生效）
volumes:
  - ./src:/app/src
```

volumes 的两种类型对比：

命名卷	postgres_data:/var/lib/...	Docker 管理的目录	数据库数据持久化
绑定挂载	./src:/app/src	你项目的实际目录	开发时热更新代码

restart:

[yamll]

```
restart: always      # 容器崩溃后自动重启
restart: unless-stopped # 除非手动停止，否则一直重启
```

networks: (进阶，了解即可)

[yamll]

```
# 默认情况下，docker-compose 自动创建一个网络，所有服务都在这个网络里
# 服务间通过服务名互相访问，不需要额外配置网络

# 如果你需要自定义网络（一般不需要）：
networks:
  agent-net:
    driver: bridge
```

6.4 完整的 Multi-Agent 项目 docker-compose.yml 示例

[yamll]

```
services:
  # FastAPI 后端应用
  fastapi-app:
    build: ./src      # 从 ./src 目录的 Dockerfile 构建
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@db:5432/multiagent
      REDIS_URL: redis://redis:6379/0
      MILVUS_HOST: milvus
      MILVUS_PORT: 19530
      OLLAMA_BASE_URL: http://ollama:11434
    depends_on:
      - db
      - redis
      - milvus
    restart: unless-stopped

  # PostgreSQL 数据库
  db:
    image: postgres:17
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: multiagent
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

  # Redis 缓存
  redis:
```

```
image: redis:7.4-alpine
ports:
  - "6379:6379"
restart: unless-stopped

# Ollama AI 服务
ollama:
  image: ollama/ollama:latest
  ports:
    - "11434:11434"
  volumes:
    - ollama_data:/root/.ollama
  restart: unless-stopped
  # deploy:          # 如果有 GPU，取消注释
  #   resources:
  #     reservations:
  #       devices:
  #         - driver: nvidia
  #           count: 1
  #           capabilities: [gpu]

# Milvus 向量数据库
milvus:
  image: milvusdb/milvus:latest
  ports:
    - "19530:19530"
  volumes:
    - milvus_data:/var/lib/milvus
  environment:
    ETCD_USE_EMBED: "true"
    ETCD_DATA_DIR: /var/lib/milvus/etcd
  command: ["milvus", "run", "standalone"]
  restart: unless-stopped

volumes:
  postgres_data:
  ollama_data:
  milvus_data:
```

6.5 docker-compose 常用命令

```
[ bash ]
```

```
# 启动所有服务（前台运行，日志直接输出到终端）
```

```
docker-compose up
```

```
# 启动所有服务（后台运行）
```

```
docker-compose up -d
```

```
# 启动后重新构建镜像
```

```
docker-compose up -d --build
```

```
# 查看运行中的服务
```

```
docker-compose ps
```

```
# 查看某个服务的日志
```

```
docker-compose logs fastapi-app
```

```
docker-compose logs -f db      # 实时跟踪
```

```
# 进入某个服务的容器
```

```
docker-compose exec fastapi-app /bin/bash
docker-compose exec db psql -U postgres -d multiagent

# 停止所有服务（不删除数据）
docker-compose stop

# 停止并删除所有容器、网络（数据卷保留）
docker-compose down

# 停止并删除所有容器、网络、数据卷（危险操作！数据会丢失）
docker-compose down -v

# 单独启动某个服务
docker-compose up -d db

# 单独停止某个服务
docker-compose stop redis
```

7. 实战：用 docker-compose 启动 PostgreSQL + Redis

7.1 创建项目

创建一个临时目录来练习：

```
[ bash ]

# 在你的项目目录下
cd E:/360MoveData/Users/why/Desktop/Agent项目/Multi-Agent

# 创建练习目录
mkdir docker-practice
cd docker-practice
```

7.2 创建 docker-compose.yml

用编辑器创建以下内容的文件 **docker-compose.yml**：

```
[ yaml ]

services:
  postgres:
    image: postgres:17
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: testuser
      POSTGRES_PASSWORD: testpass
      POSTGRES_DB: testdb
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7.4-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

7.3 启动并验证

```
[ bash ]
```

```
# 1. 启动
docker-compose up -d

# 2. 查看运行状态
docker-compose ps

# 3. 验证 PostgreSQL
docker-compose exec postgres psql -U testuser -d testdb -c "SELECT version();"

# 4. 验证 Redis
docker-compose exec redis redis-cli ping
# 应该返回 PONG

# 5. 查看日志
docker-compose logs postgres

# 6. 用本机工具连接测试（可选）
# PostgreSQL: psql -h localhost -p 5432 -U testuser -d testdb
# Redis: redis-cli -h localhost -p 6379 ping

# 7. 停止并清理
docker-compose down
```

7.4 常见错误排查

port is already allocated	端口被占用	换端口，如 "5433:5432" 或先 <code>docker-compose down</code>
service not found	服务名拼写错误	检查 <code>docker-compose.yml</code> 中的服务名
容器反复重启	配置错误	用 <code>docker-compose logs <服务名></code> 查看原因
数据库连接被拒绝	数据库还没就绪	等几秒再试，或加 <code>depends_on</code>

第三部分：FastAPI 基础

8. FastAPI 是什么

FastAPI 是一个现代的 Python Web 框架，用于构建 API。

核心特点：

- - 快：性能接近 Node.js 和 Go
- - 简单：代码量少，直观化
- - 自动文档：自动生成 Swagger UI 和 ReDoc
- - 类型安全：基于 Python 类型提示，减少 bug

类比:

- - Flask = 一个记事本, 简单但功能有限
- - Django = 一个完整的 Word, 功能全但很重
- - FastAPI = 一个现代化的 Markdown 编辑器, 轻快且好用

9. 第一个 FastAPI 应用

9.1 最小可用示例

创建文件 `app.py`:

```
[ python ]
```

```
# 导入 FastAPI 类
from fastapi import FastAPI

# 创建应用实例
app = FastAPI()

# 定义一个路由: GET /
@app.get("/")
def root():
    return {"message": "Hello World"}

# 定义一个路由: GET /health
@app.get("/health")
def health_check():
    return {"status": "ok"}
```

9.2 运行应用

```
[ bash ]
```

```
# 安装 uvicorn (FastAPI 的 ASGI 服务器)
pip install fastapi uvicorn

# 运行
uvicorn app:app --reload --host 0.0.0.0 --port 8000

# 解释:
# app:app 第一个 app 是文件名 (app.py), 第二个 app 是 FastAPI 实例名
# --reload 代码修改后自动重启 (开发时方便)
# --host 监听所有网络接口
# --port 监听 8000 端口
```

9.3 测试

```
[ bash ]
```

```
# 浏览器打开
http://localhost:8000

# 或用 curl
curl http://localhost:8000
# 返回: {"message":"Hello World"}

curl http://localhost:8000/health
# 返回: {"status":"ok"}
```

9.4 返回不同数据类型

[python]

```
from fastapi import FastAPI

app = FastAPI()

# 返回字典
@app.get("/user")
def get_user():
    return {"name": "Alice", "age": 25}

# 返回列表
@app.get("/users")
def get_users():
    return [
        {"name": "Alice", "age": 25},
        {"name": "Bob", "age": 30},
    ]

# 返回字符串
@app.get("/greet")
def greet():
    return "Hello!"
```

10. 路径参数和查询参数

10.1 路径参数 (Path Parameter)

路径参数是 URL 路径的一部分，如 `/users/123` 中的 `123`：

[python]

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id, "name": "Alice"}

@app.get("/users/{user_id}/orders")
def get_user_orders(user_id: int):
    return {"user_id": user_id, "orders": ["订单1", "订单2"]}
```

测试：

[bash]

```
curl http://localhost:8000/users/123
# 返回: {"user_id": 123, "name": "Alice"}

curl http://localhost:8000/users/456/orders
# 返回: {"user_id": 456, "orders": ["订单1", "订单2"]}
```

类型自动校验：

[python]

```
@app.get("/items/{item_id}")
def get_item(item_id: int):    # 必须是整数
    return {"item_id": item_id}
```

如果访问 `/items/abc`，FastAPI 会自动返回 422 错误，因为 `abc` 不是整数。

10.2 查询参数 (Query Parameter)

查询参数是 URL 中 `?` 后面的部分，如 `/items?skip=0&limit=10`：

[python]

```
@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

测试：

[bash]

```
curl http://localhost:8000/items
# 返回: {"skip": 0, "limit": 10} (使用默认值)

curl "http://localhost:8000/items?skip=20&limit=5"
# 返回: {"skip": 20, "limit": 5}

curl "http://localhost:8000/items?limit=abc"
# 返回 422 错误 (abc 不是整数)
```

10.3 路径参数 + 查询参数组合

[python]

```
@app.get("/users/{user_id}/items")
def get_user_items(user_id: int, skip: int = 0, limit: int = 10):
    return {
        "user_id": user_id,
        "skip": skip,
        "limit": limit,
    }
```

测试：

[bash]

```
curl "http://localhost:8000/users/1/items?skip=5&limit=2"
# 返回: {"user_id": 1, "skip": 5, "limit": 2}
```

11. 请求体与 Pydantic 模型

11.1 什么是请求体

POST/PUT 请求通常携带 JSON 数据，这就是请求体。FastAPI 用 Pydantic 模型 来定义请求体的结构。

11.2 Pydantic 模型基础

[python]

```
from pydantic import BaseModel

# 定义一个数据模型
class User(BaseModel):
    name: str          # 必须，字符串
    age: int            # 必须，整数
    email: str          # 必须，字符串
    is_active: bool = True # 可选，默认 True
```

11.3 在路由中使用

[python]

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    description: str | None = None # 可选字段
    tags: list[str] = []          # 列表, 默认空

# POST 请求, 接收 Item 数据
@app.post("/items")
def create_item(item: Item):
    # item 是 Pydantic 模型实例, 可以直接当字典用
    return {
        "message": "创建成功",
        "item": item.model_dump() # 转换为字典
    }

# PUT 请求, 更新数据
@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item):
    return {
        "message": f"更新物品 {item_id}",
        "item": item.model_dump()
    }
```

测试:

[bash]

```
curl -X POST http://localhost:8000/items \
-H "Content-Type: application/json" \
-d '{"name": "手机", "price": 2999.0, "tags": ["电子", "通讯"]}'

# 返回:
# {"message": "创建成功", "item": {"name": "手机", "price": 2999.0, "description": null, "tags": ["电子", "通讯"]}}
```

11.4 数据校验

Pydantic 会自动校验数据:

[bash]

```
# 缺少必填字段 name
curl -X POST http://localhost:8000/items \
-H "Content-Type: application/json" \
-d '{"price": 100}'

# 返回 422 错误:
# {"detail":[{"type": "missing", "loc": ["body", "name"], "msg": "Field required"}]}

# price 类型不对 (传了字符串)
curl -X POST http://localhost:8000/items \
-H "Content-Type: application/json" \
-d '{"name": "手机", "price": "not_a_number"}'

# 返回 422 错误:
# {"detail":[{"type": "float_parsing", "loc": ["body", "price"], "msg": "Input should be a valid number"}]}
```


11.5 嵌套模型

[python]

```
class Address(BaseModel):
    city: str
    street: str

class User(BaseModel):
    name: str
    address: Address    # 嵌套另一个模型

@app.post("/users")
def create_user(user: User):
    return {"name": user.name, "city": user.address.city}
```

12. 异步编程 async/await

12.1 什么是异步

同步：一件事做完再做下一件（排队等待）

异步：一件事在等待时，可以去做别的事（不傻等）

12.2 类比

同步（餐厅只有一个服务员）：

- 客人 A 点餐 → 服务员去厨房等菜（等了 10 分钟）→ 等菜好了 → 端给 A
- 期间客人 B、C 都没人管

异步（服务员不等菜）：

- 客人 A 点餐 → 下单给厨房 → 去服务客人 B
- 菜好了（通知）→ 端给 A
- 同样一个服务员，能服务更多客人

12.3 在 FastAPI 中的写法

[python]

```
from fastapi import FastAPI
import asyncio

app = FastAPI()

# 同步路由（普通 def）
@app.get("/sync")
def sync_endpoint():
    return {"type": "sync"}

# 异步路由（async def）
@app.get("/async")
async def async_endpoint():
    return {"type": "async"}
```

什么时候用 async?

调用数据库	是 (IO 操作)
调用外部 API	是 (网络请求)
读写文件	是 (IO 操作)
调用 LLM/Ollama	是 (网络请求)
纯计算、数据处理	否
简单的返回	都可以

实际应用：异步调用数据库

```
[ python ]
```

```
import asyncio

@app.get("/slow-data")
async def get_slow_data():
    # 模拟慢的数据库调用 (等待 2 秒)
    await asyncio.sleep(2)
    return {"data": "done"}
```

在我们的 Multi-Agent 项目中，几乎所有 Agent 调用都应该是 async 的：

```
[ python ]
```

```
async def call_ollama(prompt: str) -> str:
    # 异步调用 Ollama API
    async with httpx.AsyncClient() as client:
        response = await client.post("http://ollama:11434/api/generate", ...)
    return response.text
```

13. 依赖注入 Depends

13.1 什么是依赖注入

简单说：让函数自动获取它需要的东西，而不是每次手动传。

不用 Depends（每次都手动传）：

```
[ python ]
```

```
def get_db():
    # 复杂逻辑获取数据库连接
    return db_connection

@app.get("/users")
def get_users(db = None):
    db = get_db() # 手动获取
    return db.query(...)

@app.get("/items")
def get_items(db = None):
    db = get_db() # 又要手动获取...
```

用 Depends（自动注入）：

```
[ python ]
```

```

from fastapi import Depends

def get_db():
    db = "数据库连接"
    try:
        yield db        # 提供给路由使用
    finally:
        print("关闭连接") # 请求结束后清理资源

@app.get("/users")
def get_users(db = Depends(get_db)): # 自动注入
    return {"db": db}

@app.get("/items")
def get_items(db = Depends(get_db)): # 自动注入
    return {"db": db}

```

13.2 实际应用：数据库会话依赖

[python]

```

from fastapi import Depends, FastAPI
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session

DATABASE_URL = "postgresql://user:pass@db:5432/mydb"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

app = FastAPI()

@app.get("/users")
def list_users(db: Session = Depends(get_db)):
    users = db.query(User).all()
    return users

```

14. 自动文档 Swagger UI

FastAPI 的最大卖点之一：写完代码就有文档，不用手写。

14.1 查看文档

运行 FastAPI 应用后：

```

# Swagger UI (交互式, 可以测试 API)
http://localhost:8000/docs

# ReDoc (只读文档, 更美观)
http://localhost:8000/redoc

# OpenAPI JSON (机器可读的 API 定义)

```

http://localhost:8000/openapi.json

14.2 让文档更完善

[python]

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI(
    title="我的 API",
    description="这是 API 的描述",
    version="1.0.0",
)

class Item(BaseModel):
    name: str = Field(..., description="物品名称", example="手机")
    price: float = Field(..., description="价格", example=2999.0)
    description: str | None = Field(None, description="物品描述")

@app.post("/items", tags=["物品管理"], summary="创建物品")
def create_item(item: Item):
    """
    创建一个新的物品。

    - **name**: 物品名称（必填）
    - **price**: 物品价格（必填）
    - **description**: 物品描述（可选）
    """
    return {"item": item.model_dump()}
```

效果：

- - [/docs](#) 页面会显示函数文档中的描述
- - 参数说明、示例值都会在 Swagger UI 中展示

第四部分：综合实战

15. 综合实战：完整项目

目标：用 FastAPI + PostgreSQL + Docker Compose 搭建一个物品管理 CRUD API。

这是你开始 Multi-Agent 项目前必须能独立完成任务。

15.1 项目结构

```
crud-project/
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── app/
│   ├── __init__.py
│   ├── main.py      # FastAPI 应用入口
│   ├── database.py  # 数据库连接
│   └── models.py     # SQLAlchemy 数据模型
```

| — schemas.py # Pydantic 请求/响应模型

15.2 创建文件

requirements.txt

```
fastapi==0.115.0
uvicorn==0.30.0
sqlalchemy==2.0.35
psycopg2-binary==2.9.9
pydantic==2.9.0
pydantic-settings==2.5.0
```

docker-compose.yml

[yml]

```
services:
  db:
    image: postgres:17
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: crud_db
    volumes:
      - postgres_data:/var/lib/postgresql/data

  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@db:5432/crud_db
    depends_on:
      - db
    volumes:
      - ./app:/app/app # 绑定挂载：改代码自动生效

volumes:
  postgres_data:
```

Dockerfile

[dockerfile]

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

app/__init__.py

(空文件即可)

app/database.py

[python]

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
import os
```

```
# 从环境变量读取数据库连接地址
DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@db:5432/crud_db")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    """获取数据库会话的依赖函数"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

app/models.py

[python]

```
from sqlalchemy import Column, Integer, String, Float
from app.database import Base

class Item(Base):
    __tablename__ = "items"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    price = Column(Float, nullable=False)
    description = Column(String(500), nullable=True)
```

app/schemas.py

[python]

```
from pydantic import BaseModel, Field

class ItemCreate(BaseModel):
    """创建物品时的请求体"""
    name: str = Field(..., min_length=1, max_length=100)
    price: float = Field(..., gt=0)
    description: str | None = None

class ItemResponse(BaseModel):
    """返回给客户端的物品数据"""
    id: int
    name: str
    price: float
    description: str | None

class Config:
    from_attributes = True # 允许从 SQLAlchemy 模型转换
```

app/main.py

[python]

```
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import engine, get_db, Base
from app.models import Item
from app.schemas import ItemCreate, ItemResponse
```

```

# 启动时创建数据库表
Base.metadata.create_all(bind=engine)

app = FastAPI(title="物品管理 API", version="1.0.0")

# ----- 增 (Create) -----
@app.post("/items", response_model=ItemResponse, status_code=201)
def create_item(item: ItemCreate, db: Session = Depends(get_db)):
    """创建新物品"""
    db_item = Item(
        name=item.name,
        price=item.price,
        description=item.description,
    )
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item

# ----- 查 (Read) -----
@app.get("/items", response_model=list[ItemResponse])
def list_items(skip: int = 0, limit: int = 10, db: Session = Depends(get_db)):
    """获取物品列表"""
    items = db.query(Item).offset(skip).limit(limit).all()
    return items

@app.get("/items/{item_id}", response_model=ItemResponse)
def get_item(item_id: int, db: Session = Depends(get_db)):
    """获取单个物品"""
    item = db.query(Item).filter(Item.id == item_id).first()
    if not item:
        raise HTTPException(status_code=404, detail="物品不存在")
    return item

# ----- 改 (Update) -----
@app.put("/items/{item_id}", response_model=ItemResponse)
def update_item(item_id: int, item_data: ItemCreate, db: Session = Depends(get_db)):
    """更新物品信息"""
    item = db.query(Item).filter(Item.id == item_id).first()
    if not item:
        raise HTTPException(status_code=404, detail="物品不存在")

    item.name = item_data.name
    item.price = item_data.price
    item.description = item_data.description
    db.commit()
    db.refresh(item)
    return item

# ----- 删 (Delete) -----
@app.delete("/items/{item_id}", status_code=204)
def delete_item(item_id: int, db: Session = Depends(get_db)):
    """删除物品"""
    item = db.query(Item).filter(Item.id == item_id).first()
    if not item:
        raise HTTPException(status_code=404, detail="物品不存在")

    db.delete(item)
    db.commit()

```

15.3 启动和测试

```
[ bash ]
```

```
# 进入项目目录
cd crud-project

# 启动所有服务
docker-compose up -d --build

# 查看状态
docker-compose ps

# 查看日志 (如有问题)
docker-compose logs api
docker-compose logs db

# 测试 API
curl http://localhost:8000/items
# 返回: [] (空列表)

# 创建物品
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "手机", "price": 2999.0, "description": "一台智能手机"}'

# 查询列表
curl http://localhost:8000/items

# 查询单个
curl http://localhost:8000/items/1

# 更新
curl -X PUT http://localhost:8000/items/1 \
  -H "Content-Type: application/json" \
  -d '{"name": "旗舰手机", "price": 5999.0, "description": "旗舰版"}'

# 删除
curl -X DELETE http://localhost:8000/items/1

# 打开文档测试
# 浏览器: http://localhost:8000/docs

# 停止
docker-compose down
```

15.4 你通过这个实战掌握了什么

完成这个实战后，你已经掌握了：

- - [] Docker 基础命令
- - [] 编写 Dockerfile
- - [] 编写 docker-compose.yml
- - [] 端口映射、环境变量、数据卷
- - [] FastAPI 路由定义
- - [] Pydantic 模型校验

- - [] SQLAlchemy ORM
- - [] 依赖注入 Depends
- - [] 异步基础
- - [] Swagger UI 文档

这些正好是开始 Multi-Agent 项目 Phase 1 所需的全部基础。

下一步

完成上述学习后，你可以：

- 开始项目的 Phase 1（基础设施搭建）
- 学习 LangGraph 的 StateGraph 概念
- 学习 Ollama 的基本使用

有什么不清楚的地方，随时问我。